

## PPL - Assignment 4: Theoretical Questions

### Question 1 - Lazy lists

---

#### a. Equivalence criterion for two lazy lists

1. **Base Case (Empty Lists):** If both lists are empty, they are equivalent
2. **Inductive Step (Non-Empty Lists):** If both lists are non-empty, they are equivalent if and only if their heads (current elements) are strictly equal, and the Tails of the lists (obtained by evaluating their tails) are also equivalent

\*Note: If one list is empty and the other is not, they are not equivalent

#### b. Equivalence proof for even-square-1 and even-square-2

Since both procedures process the infinite stream generated by `integers-from(0)`, neither list is ever empty. Therefore, the "Empty Lists" case from our definition in section (a) is trivially inapplicable. To prove that the lists are equivalent, we apply the "Non-Empty Lists" criterion and show that for any integer  $n \geq 0$ , their heads are strictly equal and the Tails of the lists (obtained by evaluating their tails) are also equivalent.

We prove this by induction on the integer  $n$ :

1. **Base Case ( $n = 0$ ):** Since 0 is an even number, both procedures evaluate as follows:
  - **Heads:** In `even-square-1`, 0 is squared to  $0^2 = 0$ , which is even and passes the filter, so the head is 0. In `even-square-2`, 0 passes the filter and is then squared to 0, so the head is 0. Their heads are strictly equal.
  - **Tails:** Evaluating their tails forces the deferred computation, moving both procedures synchronously to process the next integer,  $n = 1$ . By induction, these Tails are equivalent.
2. **Inductive Step:** Assuming the lists are equivalent up to an integer  $n$ , we evaluate the next step based on the parity of  $n$ :
  - **Case 1 -  $n$  is even:**
    - **Heads:** In `even-square-1`,  $n$  is squared to  $n^2$  (which is even) and passes the filter. In `even-square-2`,  $n$  passes the filter and is then squared to  $n^2$ . Thus, both heads are equal to  $n^2$ .

- **Tails:** Evaluating their tails forces the next step, moving both procedures to process the next integer,  $n + 1$ . By the induction hypothesis, these remaining portions are equivalent.
- **Case 2 -  $n$  is odd:** Both procedures synchronously skip the current element and immediately reduce to processing the next integer ( $n + 1$ ):
  - even-square-1 computes  $n^2$  (which is odd) and the filter skips it, moving to  $n + 1$ .
  - even-square-2 filters out  $n$  immediately because it is odd, moving to  $n + 1$ .

*Note: Since both procedures align perfectly on the base case, match on every even step, and synchronously skip every odd step, they generate the exact same infinite sequence of values (0, 4, 16, 36, ...). Therefore, even-square-1 and even-square-2 are equivalent.*

## Question 2 - CPS

---

### a. Equivalence criterion for a procedure and its Success-Fail-Continuations (SFC) version

To define the equivalence, we specify the types of the original procedure  $f$  and its SFC version  $f_{sfc}$ :

- $f : T_1 \rightarrow T_2 \cup \{\text{'fail'}\}$
- $f_{sfc} : T_1 \times (T_2 \rightarrow T_3) \times (\text{Empty} \rightarrow T_4) \rightarrow T_3 \cup T_4$

The procedures  $f$  and  $f_{sfc}$  are equivalent if and only if for any input  $x \in T_1$  and continuations  $\text{succ}$  and  $\text{fail}$ , the following conditions hold:

1. **Success Case:** If the original procedure succeeds and returns a value  $v \neq \text{'fail'}$ , the SFC version applies the  $\text{succ}$  continuation to  $v$ : if  $f(x) = v \implies f_{sfc}(x, \text{succ}, \text{fail}) = \text{succ}(v)$
2. **Failure Case:** If the original procedure fails and returns 'fail', the SFC version applies the  $\text{fail}$  continuation: if  $f(x) = \text{'fail'} \implies f_{sfc}(x, \text{succ}, \text{fail}) = \text{fail}()$

### d. Equivalence proof for get-value and get-value\$

We prove that  $\text{get-value}(f)$  and  $\text{get-value}(f_{\text{sfc}})$  are equivalent according to our definition by induction on the structure of  $\text{assoc-list}$ :

1. **Base Case (Empty List):** If  $\text{assoc-list}$  is empty ( $()$ ), both procedures evaluate as follows:

- The original procedure returns the symbol 'fail:  $f('(), \text{key}) = \text{'fail}$
- The SFC version evaluates the fail continuation:  $f_{sfc}('(), \text{key}, \text{succ}, \text{fail}) = \text{fail}()$

2. **Inductive Step (Non-Empty List):** Assuming equivalence holds for the rest of the list (cdr assoc-list), we evaluate the current step based on whether the key matches the current pair's key (caar assoc-list):

- **Case 1 - The key matches (key == caar assoc-list):**
  - The original procedure succeeds and directly returns the corresponding value  $v$ :  
 $f(\text{assoc-list}, \text{key}) = \text{cdar assoc-list} = v$
  - The SFC version evaluates the succ continuation on that same value:  
 $f_{sfc}(\text{assoc-list}, \text{key}, \text{succ}, \text{fail}) = \text{succ}(\text{cdar assoc-list}) = \text{succ}(v)$
- **Case 2 - The key does not match (key != caar assoc-list):**
  - Both procedures bypass the current element and reduce directly to processing the tail of the list (cdr assoc-list):  
 $f(\text{assoc-list}, \text{key}) = f(\text{cdr assoc-list}, \text{key})$   
 $f_{sfc}(\text{assoc-list}, \text{key}, \text{succ}, \text{fail}) = f_{sfc}(\text{cdr assoc-list}, \text{key}, \text{succ}, \text{fail})$
  - By the induction hypothesis, these remaining computations are equivalent.

Note: Since both procedures match perfectly on the base case, align on immediate matches, and synchronously delegate to the tail when a key mismatch occurs, they are structurally equivalent for any input.